4. Entregable 04 · Diagramas UML

4.1 Diagrama de clases completo (código PlantUML)

Incluye las 10 entidades del modelo de dominio original más las agregadas **(propuesto)**: `LineaVenta`, `TransferenciaInventario`, `Pago`, `Anticipo`, `RegistroAuditoria` y las interfaces de repositorio.

```
@startuml ClasesDominio_TecnoStock
skinparam classAttributeIconSize 0
skinparam linetype ortho

enum Rol { Vendedor; Cajero; Supervisor; Almacen }
enum MetodoPago { Efectivo; Tarjeta; Transferencia }
enum EstadoApartado { Activo; Completado; Cancelado; Vencido }
enum EstadoSerial { Disponible; Vendido; Devuelto; Danado }
enum EstadoVenta { EnProceso; Pagada; Cancelada }
enum EstadoDevolucion { Solicitada; EnInspeccion; Aprobada; Rechazada; Procesada
}
enum TipoComprobante { Venta; Apartado; Devolucion }
enum TipoResolucion { Reembolso; Cambio }
enum TratamientoDiferencia { PagarDiferencia; CreditoCliente; CambioDirecto }
enum EstadoInspeccion { Aceptable; Danado }
enum Categoria { Computadora; Celular; Tablet; Accesorio; Componente }

class Producto <<Aggregate Root>> {
  - sku: SKU
  - nombre: string
  - categoria: Categoria
  - precio: Dinero
  - requiereSerial: bool
  - politicaDevolucion: PoliticaDevolucion
}
class PoliticaDevolucion <<VO>> {
  + plazoDias: int = 30
  + requiereComprobante: bool
}
class Cliente <<Aggregate Root>> {
  - documento: string
  - nombre: string
  - telefono: string
  - email: string
}
class Empleado <<Aggregate Root>> {
  - id: string
  - nombre: string
  - rol: Rol
  - sucursalId: string
  + puedeAprobarExcepciones(): bool
}
class Sucursal <<Aggregate Root>> {
  - codigo: string
  - nombre: string
```

```
  - ubicacion: string
}

class Venta <<Aggregate Root>> {
  - folio: Folio
  - fechaHora: DateTime
  - clienteId: string
  - vendedorId: string
  - cajeroId: string
  - sucursalId: string
  - estado: EstadoVenta
  - fechaVencimientoDevolucion: Date
  + agregarLinea(sku, cantidad, serial?)
  + registrarPago(pago: Pago)
  + confirmar(): VentaRegistrada
  + cancelar(motivo)
  + montoTotal(): Dinero
  + dentroDePlazoDevolucion(hoy): bool
}
class LineaVenta <<Entity>> {
  - sku: SKU
  - cantidad: int
  - precioUnitario: Dinero
  - serial: ValorSerie
  + subtotal(): Dinero
}
class Pago <<VO>> {
  + metodo: MetodoPago
  + monto: Dinero
  + montoRecibido: Dinero
  + datosElectronicos: DatosPagoElectronico
  + cambio(): Dinero
}
class DatosPagoElectronico <<VO>> {
  + ultimos4: string
  + banco: string
  + referencia: string
}
class Comprobante <<Aggregate Root>> {
  - folio: Folio
  - tipo: TipoComprobante
  - contenido: string
  - impreso: bool
  - timestamp: DateTime
  + marcarImpreso()
}

class Apartado <<Aggregate Root>> {
  - id: string
  - fechaCreacion: Date
  - clienteId: string
  - sucursalId: string
  - sku: SKU
  - cantidad: int
  - precioTotal: Dinero
```

```
  - anticipo: Anticipo
  - fechaVencimiento: Date
  - estado: EstadoApartado
  + {static} crear(cliente, producto, cantidad, anticipo, hoy)
  + saldoPendiente(): Dinero
  + completar(pagoSaldo): ApartadoCompletado
  + cancelarPorVencimiento(hoy): ApartadoCancelado
  + estaVencido(hoy): bool
  + proximoAVencer(hoy): bool
}
class Anticipo <<VO>> {
  + monto: Dinero
  + {static} validarMinimo25(monto, precioTotal)
}

class Devolucion <<Aggregate Root>> {
  - id: string
  - ventaFolio: Folio
  - clienteId: string
  - sku: SKU
  - serial: ValorSerie
  - estadoInspeccion: EstadoInspeccion
  - aprobacion: AprobacionSupervisor
  - estado: EstadoDevolucion
  - motivoRechazo: string
  + registrarInspeccion(estado)
  + aplicarAprobacion(aprobacion)
  + rechazar(motivo)
  + resolverReembolso(): Reembolso
  + resolverCambio(nuevoSku, tratamiento): Cambio
}
class AprobacionSupervisor <<VO>> {
  + supervisorId: string
  + aprobada: bool
  + motivo: string
}
class Reembolso <<VO>> {
  + monto: Dinero
  + metodoOriginal: MetodoPago
}
class Cambio <<VO>> {
  + nuevoSku: SKU
  + diferencia: Dinero
  + tratamiento: TratamientoDiferencia
}

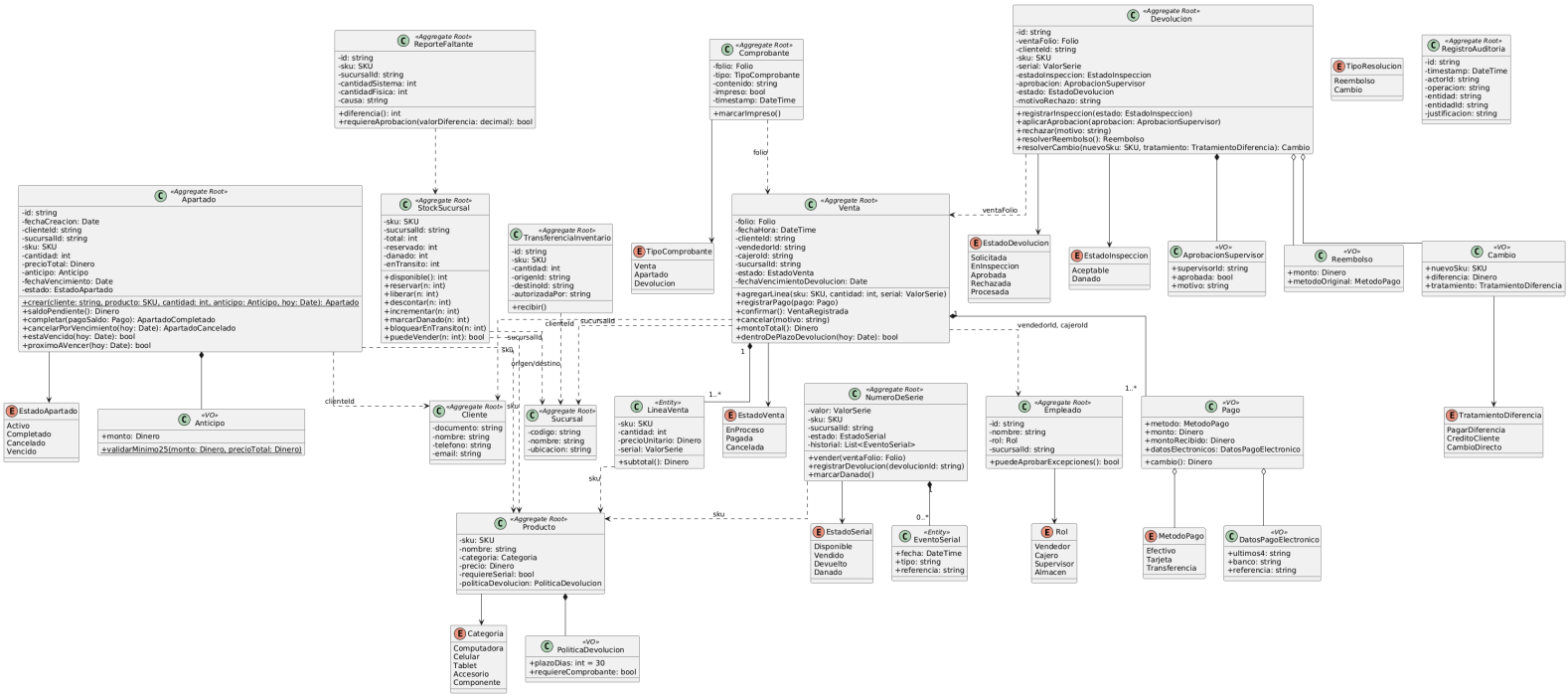
class StockSucursal <<Aggregate Root>> {
  - sku: SKU
  - sucursalId: string
  - total: int
  - reservado: int
  - danado: int
  - enTransito: int
  + disponible(): int
  + reservar(n)
```

```
  + liberar(n)
  + descontar(n)
  + incrementar(n)
  + marcarDanado(n)
  + bloquearEnTransito(n)
  + puedeVender(n): bool
}
class NumeroDeSerie <<Aggregate Root>> {
  - valor: ValorSerie
  - sku: SKU
  - sucursalId: string
  - estado: EstadoSerial
  - historial: List<EventoSerial>
  + vender(ventaFolio)
  + registrarDevolucion(devolucionId)
  + marcarDanado()
}
class EventoSerial <<Entity>> {
  + fecha: DateTime
  + tipo: string
  + referencia: string
}
class TransferenciaInventario <<Aggregate Root>> {
  - id: string
  - sku: SKU
  - cantidad: int
  - origenId: string
  - destinoId: string
  - autorizadaPor: string
  + recibir()
}
class ReporteFaltante <<Aggregate Root>> {
  - id: string
  - sku: SKU
  - sucursalId: string
  - cantidadSistema: int
  - cantidadFisica: int
  - causa: string
  + diferencia(): int
  + requiereAprobacion(valorDiferencia): bool
}
class RegistroAuditoria <<Aggregate Root>> {
  - id: string
  - timestamp: DateTime
  - actorId: string
  - operacion: string
  - entidad: string
  - entidadId: string
  - justificacion: string
}

Venta "1" *-- "1..*" LineaVenta
Venta "1" *-- "1..*" Pago
Pago o-- MetodoPago
Pago o-- DatosPagoElectronico
```

```
Venta --> EstadoVenta
Apartado *-- Anticipo
Apartado --> EstadoApartado
Devolucion *-- AprobacionSupervisor
Devolucion o-- Reembolso
Devolucion o-- Cambio
Devolucion --> EstadoDevolucion
Devolucion --> EstadoInspeccion
Cambio --> TratamientoDiferencia
NumeroDeSerie "1" *-- "0..*" EventoSerial
NumeroDeSerie --> EstadoSerial
Producto *-- PoliticaDevolucion
Producto --> Categoria
Empleado --> Rol
Comprobante --> TipoComprobante

Venta ..> Cliente : clienteId
Venta ..> Empleado : vendedorId, cajeroId
Venta ..> Sucursal : sucursalId
LineaVenta ..> Producto : sku
Apartado ..> Cliente : clienteId
Apartado ..> Producto : sku
Devolucion ..> Venta : ventaFolio
StockSucursal ..> Producto : sku
StockSucursal ..> Sucursal : sucursalId
NumeroDeSerie ..> Producto : sku
TransferenciaInventario ..> Sucursal : origen/destino
ReporteFaltante ..> StockSucursal
Comprobante ..> Venta : folio
@enduml
```



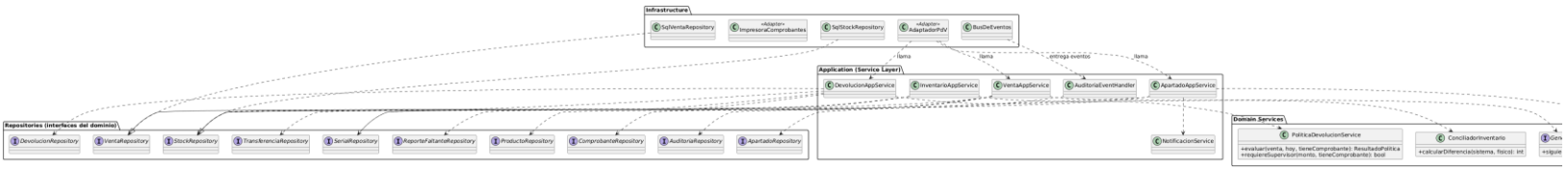
4.2 Capa de aplicación, servicios de dominio y repositorios (PlantUML)

```
@startuml ClasesAplicacion_TecnoStock
skinparam classAttributeIconSize 0

package "Application (Service Layer)" {
  class VentaAppService
  class ApartadoAppService
  class DevolucionAppService
  class InventarioAppService
  class AuditoriaEventHandler
  class NotificacionService
}
package "Domain Services" {
  class PoliticaDevolucionService {
    + evaluar(venta, hoy, tieneComprobante): ResultadoPolitica
    + requiereSupervisor(monto, tieneComprobante): bool
  }
  class CalculadoraAnticipo {
    + minimo(precioTotal): Dinero
  }
  interface GeneradorFolio {
    + siguiente(): Folio
  }
  class ConciliadorInventario {
    + calcularDiferencia(sistema, fisico): int
  }
}
package "Repositories (interfaces del dominio)" {
  interface VentaRepository
  interface ApartadoRepository
  interface DevolucionRepository
  interface StockRepository
  interface SerialRepository
  interface ProductoRepository
  interface ComprobanteRepository
  interface AuditoriaRepository
  interface TransferenciaRepository
  interface ReporteFaltanteRepository
}
package "Infrastructure" {
  class SqlVentaRepository
  class SqlStockRepository
  class AdaptadorPdV <<Adapter>>
  class ImpresoraComprobantes <<Adapter>>
  class BusDeEventos
}

VentaAppService ..> VentaRepository
VentaAppService ..> StockRepository
VentaAppService ..> SerialRepository
VentaAppService ..> ProductoRepository
VentaAppService ..> ComprobanteRepository
VentaAppService ..> GeneradorFolio
ApartadoAppService ..> ApartadoRepository
```

```
ApartadoAppService ..> StockRepository
ApartadoAppService ..> CalculadoraAnticipo
ApartadoAppService ..> NotificacionService
DevolucionAppService ..> DevolucionRepository
DevolucionAppService ..> VentaRepository
DevolucionAppService ..> PoliticaDevolucionService
DevolucionAppService ..> StockRepository
DevolucionAppService ..> SerialRepository
InventarioAppService ..> StockRepository
InventarioAppService ..> TransferenciaRepository
InventarioAppService ..> ReporteFaltanteRepository
InventarioAppService ..> ConciliadorInventario
AuditoriaEventHandler ..> AuditoriaRepository
SqlVentaRepository ..|> VentaRepository
SqlStockRepository ..|> StockRepository
AdaptadorPdV ..> VentaAppService : llama
AdaptadorPdV ..> ApartadoAppService : llama
AdaptadorPdV ..> DevolucionAppService : llama
BusDeEventos ..> AuditoriaEventHandler : entrega eventos
@enduml
```



4.3 Diagrama de secuencia 1 · Venta con número de serie y pago

Por qué este flujo: es el más completo de la rama "Venta normal" del BPMN y cubre US01, US02, US03, US05, US12, US13/US14, N1, N2 y N7. Incluye el bloqueo por producto RESERVADO y el reintento de pago ("¿Reintentar?" del BPMN).

```
@startuml Seq1_VentaConSerial
autonumber
actor Vendedor
actor Cajero
participant "PdV Windows\n(AdaptadorPdV)" as PDV
participant VentaAppService as VS
participant "Venta\n<<Aggregate>>" as V
participant "StockSucursal\n<<Aggregate>>" as ST
participant "NumeroDeSerie\n<<Aggregate>>" as NS
participant GeneradorFolio as GF
participant ComprobanteRepository as CR
participant "AuditoriaEventHandler" as AU

Vendedor -> PDV : busca producto (codigo / nombre)
PDV -> VS : ConsultarDisponibilidad(codigo)
VS --> PDV : disponible y apartado por sucursal (US01)

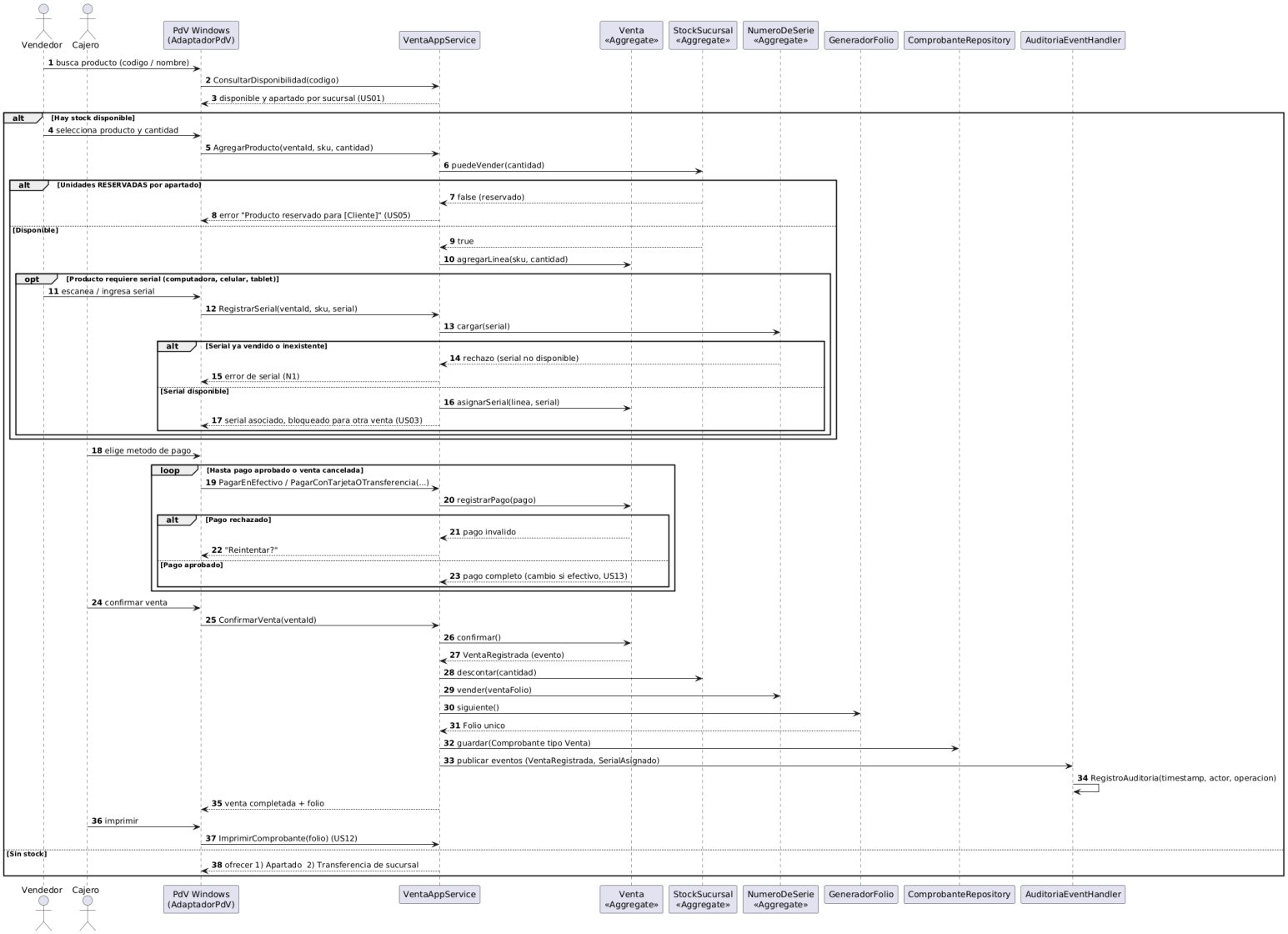
alt Hay stock disponible
  Vendedor -> PDV : selecciona producto y cantidad
```

```
  PDV -> VS : AgregarProducto(ventaId, sku, cantidad)
  VS -> ST : puedeVender(cantidad)
  alt Unidades RESERVADAS por apartado
    ST --> VS : false (reservado)
    VS --> PDV : error "Producto reservado para [Cliente]" (US05)
  else Disponible
    ST --> VS : true
    VS -> V : agregarLinea(sku, cantidad)
    opt Producto requiere serial (computadora, celular, tablet)
      Vendedor -> PDV : escanea / ingresa serial
      PDV -> VS : RegistrarSerial(ventaId, sku, serial)
      VS -> NS : cargar(serial)
      alt Serial ya vendido o inexistente
        NS --> VS : rechazo (serial no disponible)
        VS --> PDV : error de serial (N1)
      else Serial disponible
        VS -> V : asignarSerial(linea, serial)
        VS --> PDV : serial asociado, bloqueado para otra venta (US03)
      end
    end
  end

  Cajero -> PDV : elige metodo de pago
  loop Hasta pago aprobado o venta cancelada
    PDV -> VS : PagarEnEfectivo / PagarConTarjetaOTransferencia(...)
    VS -> V : registrarPago(pago)
    alt Pago rechazado
      V --> VS : pago invalido
      VS --> PDV : "Reintentar?"
    else Pago aprobado
      V --> VS : pago completo (cambio si efectivo, US13)
    end
  end

  Cajero -> PDV : confirmar venta
  PDV -> VS : ConfirmarVenta(ventaId)
  VS -> V : confirmar()
  V --> VS : VentaRegistrada (evento)
  VS -> ST : descontar(cantidad)
  VS -> NS : vender(ventaFolio)
  VS -> GF : siguiente()
  GF --> VS : Folio unico
  VS -> CR : guardar(Comprobante tipo Venta)
  VS -> AU : publicar eventos (VentaRegistrada, SerialAsignado)
  AU -> AU : RegistroAuditoria(timestamp, actor, operacion)
  VS --> PDV : venta completada + folio
  Cajero -> PDV : imprimir
  PDV -> VS : ImprimirComprobante(folio) (US12)
else Sin stock
  VS --> PDV : ofrecer 1) Apartado  2) Transferencia de sucursal
end
@enduml
```

Lectura del diagrama: la validación del RESERVADO y del serial único ocurre dentro de los Aggregates (`StockSucursal`, `NumeroDeSerie`); `VentaAppService` solo coordina. Descuento de inventario, cambio de estado del serial, folio y comprobante se confirman en una sola transacción.



4.4 Diagrama de secuencia 2 · Devolución con aprobación de Supervisor y cambio

Por qué este flujo: es la rama más compleja del BPMN (comprobante, supervisor, plazo de 30 días, inspección, reembolso o cambio con diferencia). Cubre US08, US09, N5 y las reglas 5, 6, 7, 8.

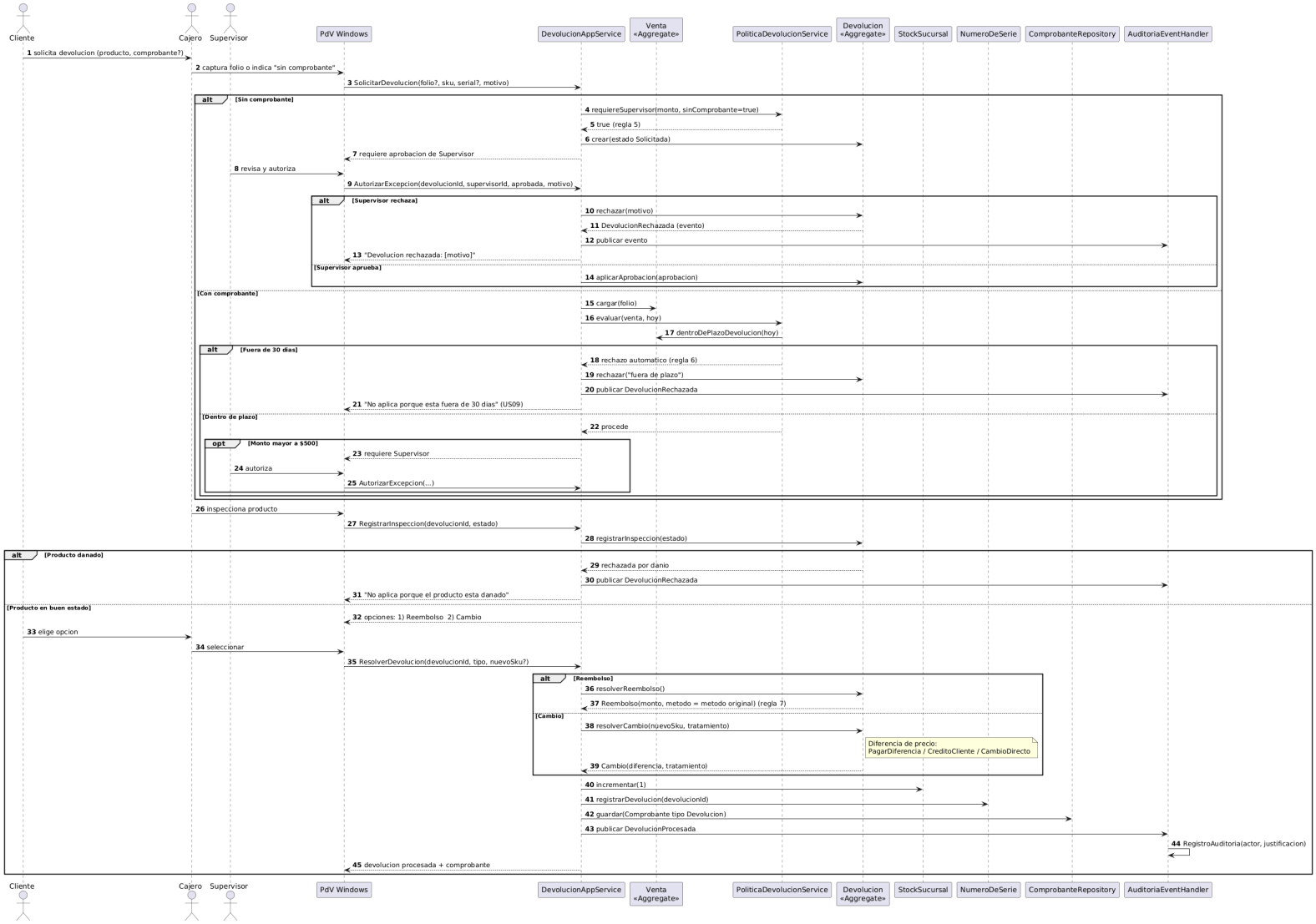
```
@startuml Seq2_Devolucion
autonumber
actor Cliente
actor Cajero
actor Supervisor
participant "PdV Windows" as PDV
participant DevolucionAppService as DS
participant "Venta\n<<Aggregate>>" as V
participant PoliticaDevolucionService as PD
participant "Devolucion\n<<Aggregate>>" as D
participant "StockSucursal" as ST
participant "NumeroDeSerie" as NS
participant ComprobanteRepository as CR
participant AuditoriaEventHandler as AU

Cliente -> Cajero : solicita devolucion (producto, comprobante?)
Cajero -> PDV : captura folio o indica "sin comprobante"
PDV -> DS : SolicitarDevolucion(folio?, sku, serial?, motivo)

alt Sin comprobante
  DS -> PD : requiereSupervisor(monto, sinComprobante=true)
  PD --> DS : true (regla 5)
  DS -> D : crear(estado Solicitada)
  DS --> PDV : requiere aprobacion de Supervisor
  Supervisor -> PDV : revisa y autoriza
  PDV -> DS : AutorizarExcepcion(devolucionId, supervisorId, aprobada, motivo)
  alt Supervisor rechaza
    DS -> D : rechazar(motivo)
    D --> DS : DevolucionRechazada (evento)
    DS -> AU : publicar evento
    DS --> PDV : "Devolucion rechazada: [motivo]"
  else Supervisor aprueba
    DS -> D : aplicarAprobacion(aprobacion)
  end
else Con comprobante
  DS -> V : cargar(folio)
  DS -> PD : evaluar(venta, hoy)
  PD -> V : dentroDePlazoDevolucion(hoy)
  alt Fuera de 30 dias
    PD --> DS : rechazo automatico (regla 6)
    DS -> D : rechazar("fuera de plazo")
    DS -> AU : publicar DevolucionRechazada
    DS --> PDV : "No aplica porque esta fuera de 30 dias" (US09)
  else Dentro de plazo
    PD --> DS : procede
    opt Monto mayor a $500
      DS --> PDV : requiere Supervisor
      Supervisor -> PDV : autoriza
      PDV -> DS : AutorizarExcepcion(...)
    end
```

```
  end
end

Cajero -> PDV : inspecciona producto
PDV -> DS : RegistrarInspeccion(devolucionId, estado)
DS -> D : registrarInspeccion(estado)
alt Producto danado
  D --> DS : rechazada por danio
  DS -> AU : publicar DevolucionRechazada
  DS --> PDV : "No aplica porque el producto esta danado"
else Producto en buen estado
  DS --> PDV : opciones: 1) Reembolso  2) Cambio
  Cliente -> Cajero : elige opcion
  Cajero -> PDV : seleccionar
  PDV -> DS : ResolverDevolucion(devolucionId, tipo, nuevoSku?)
  alt Reembolso
    DS -> D : resolverReembolso()
    D --> DS : Reembolso(monto, metodo = metodo original) (regla 7)
  else Cambio
    DS -> D : resolverCambio(nuevoSku, tratamiento)
    note right of D : Diferencia de precio:\nPagarDiferencia / CreditoCliente /
CambioDirecto
    D --> DS : Cambio(diferencia, tratamiento)
  end
  DS -> ST : incrementar(1)
  DS -> NS : registrarDevolucion(devolucionId)
  DS -> CR : guardar(Comprobante tipo Devolucion)
  DS -> AU : publicar DevolucionProcesada
  AU -> AU : RegistroAuditoria(actor, justificacion)
  DS --> PDV : devolucion procesada + comprobante
end
@enduml
```


Cliente
Cajero
Supervisor
PdV Windows
DevolucionAppService
Venta «Aggregate»
PoliticaDevolucionService
Devolucion «Aggregate»
StockSucursal
NumeroDeSerie
ComprobanteRepository
AuditoriaEventHandler
1 solicita devolucion (producto, comprobante?)
2 captura folio o indica "sin comprobante"
3 SolicitarDevolucion(folio?, sku, serial?, motivo)
alt [Sin comprobante]
4 requiereSupervisor(monto, sinComprobante=true)
5 true (regla 5)
6 crear(estado Solicitada)
7 requiere aprobacion de Supervisor
8 revisa y autoriza
9 AutorizarExcepcion(devolucionId, supervisorId, aprobada, motivo)
alt [Supervisor rechaza]
10 rechazar(motivo)
11 DevolucionRechazada (evento)
12 publicar evento
13 "Devolucion rechazada: [motivo]"
[Supervisor aprueba]
14 aplicarAprobacion(aprobacion)
[Con comprobante]
15 cargar(folio)
16 evaluar(venta, hoy)
17 dentroDePlazoDevolucion(hoy)
alt [Fuera de 30 dias]
18 rechazo automatico (regla 6)
19 rechazar("fuera de plazo")
20 publicar DevolucionRechazada
21 "No aplica porque esta fuera de 30 dias" (US09)
[Dentro de plazo]
22 procede
opt [Monto mayor a $500]
23 requiere Supervisor
24 autoriza
25 AutorizarExcepcion(...)
26 inspecciona producto
27 RegistrarInspeccion(devolucionId, estado)
28 registrarInspeccion(estado)
alt [Producto danado]
29 rechazada por daño
30 publicar DevolucionRechazada
31 "No aplica porque el producto esta danado"
[Producto en buen estado]
32 opciones: 1) Reembolso 2) Cambio
33 elige opcion
34 seleccionar
35 ResolverDevolucion(devolucionId, tipo, nuevoSku?)
alt [Reembolso]
36 resolverReembolso()
37 Reembolso(monto, metodo = metodo original) (regla 7)
[Cambio]
38 resolverCambio(nuevoSku, tratamiento)
Diferencia de precio:
PagarDiferencia / CreditoCliente / CambioDirecto
39 Cambio(diferencia, tratamiento)
40 incrementar(1)
41 registrarDevolucion(devolucionId)
42 guardar(Comprobante tipo Devolucion)
43 publicar DevolucionProcesada
44 RegistroAuditoria(actor, justificacion)
45 devolucion procesada + comprobante